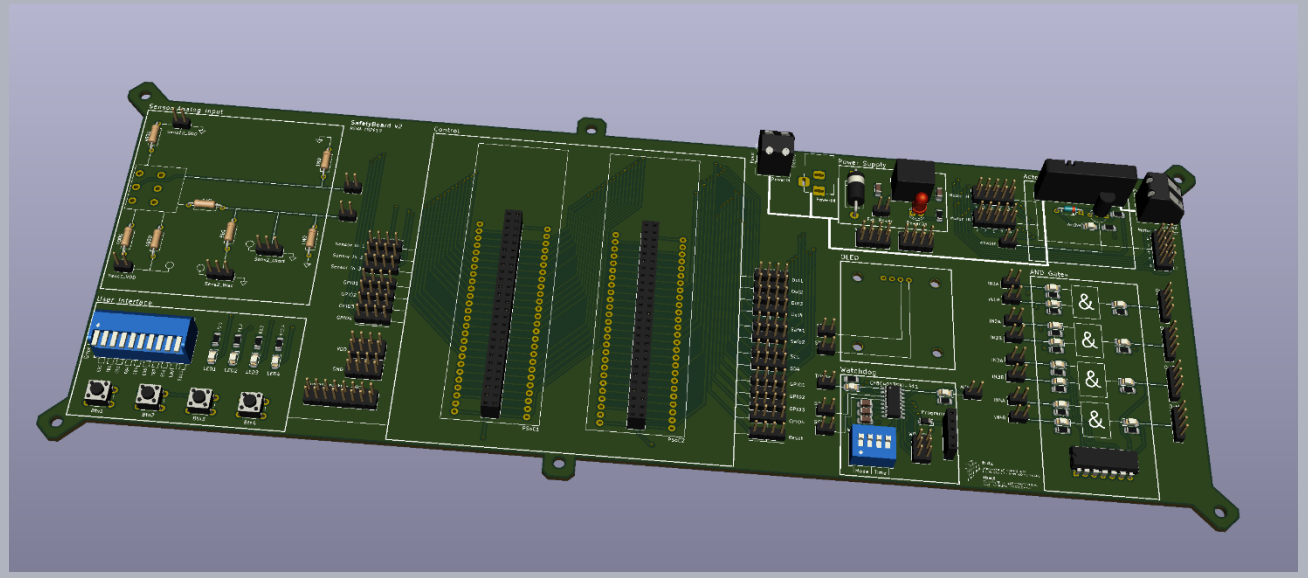
Safety in Embedded Control Systems

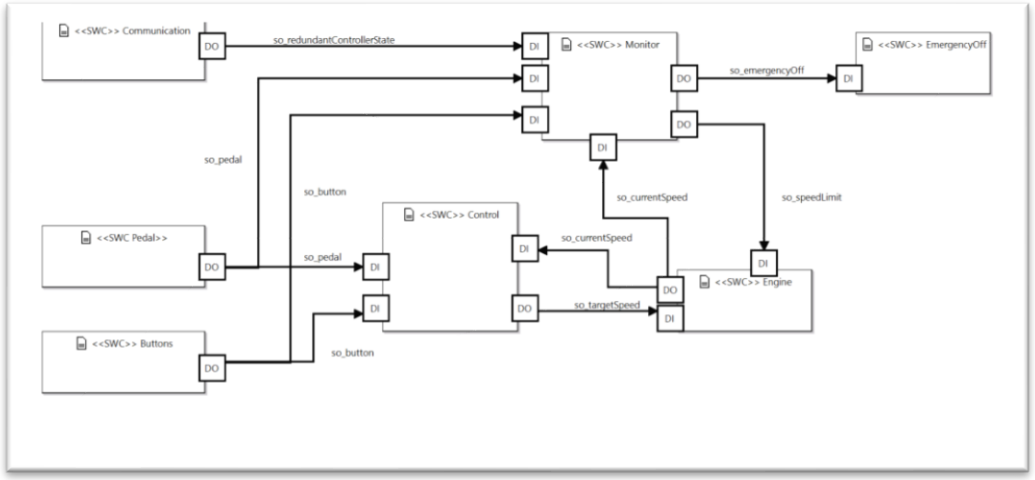
Software Architecture and Process

Prof. Dr.-Ing. Peter Fromm

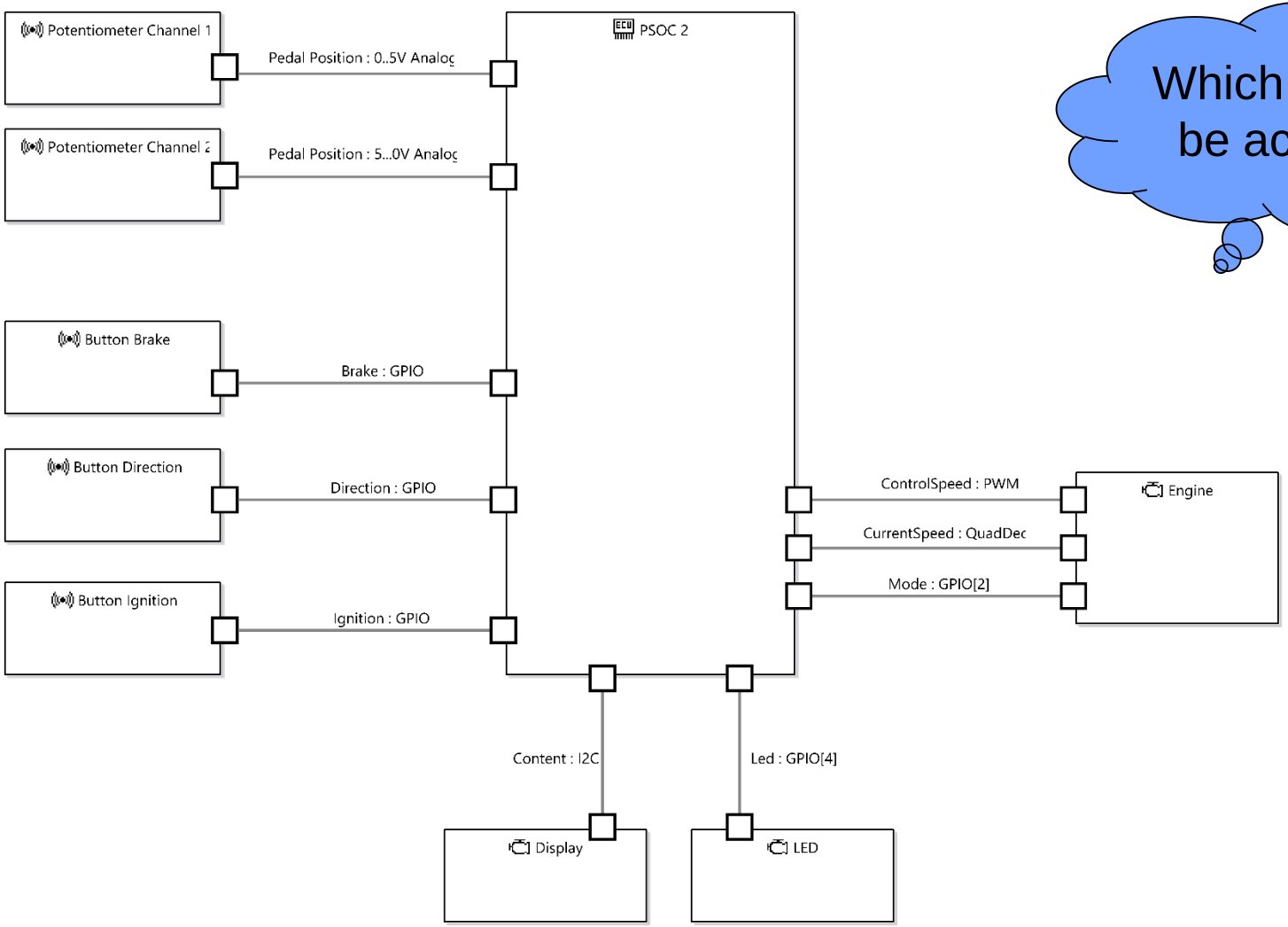


Content

- A look at the functional and hardware design
- Software Safety Requirements
- Architectural Development
  - Modelling
  - Design requirements
- Coding
  - Design and Code Guidelines
  - Misra
  - Static Code Analysis
- Testing (more to come)



Last Lecture: a first hardware architecture



Which ASIL can be achieved?

Main Requirements for a Safety Architecture

Reliable detection and handling of stochastic faults and errors (focus single faults)
E.g. hardware failures, communication errors,…

Reduction / avoidance of systematic errors.
E.g. programming bugs

In case of a critical failure: Transition into a safe state.

Safety Mechanisms

Component Reliability

- MTTF
- Process

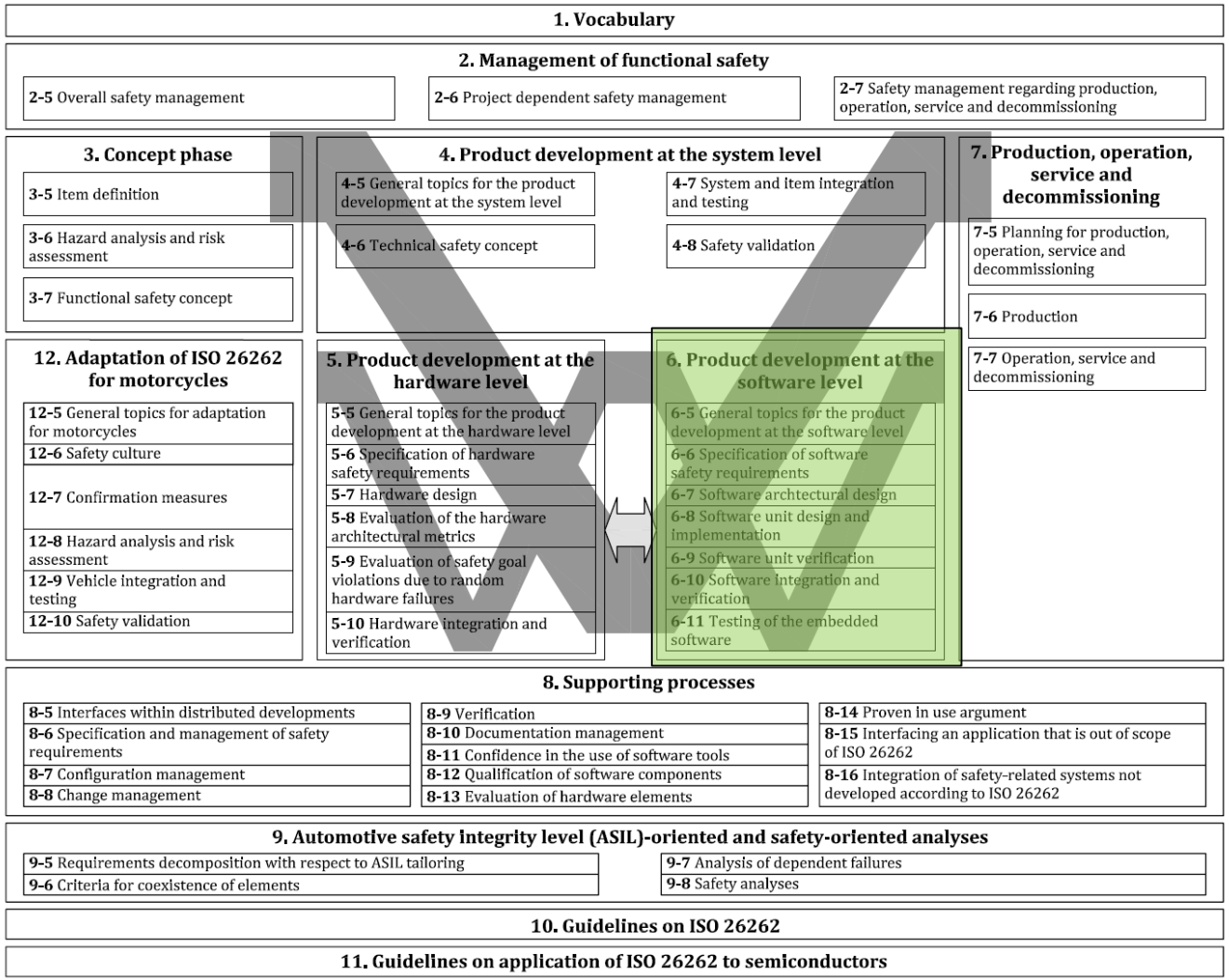
Architecture

- Redundancy
- Freedom from Interferance
- Common Cause Failure
- Safe State

Fault Detection

- Diagnostic Coverage
- Error Handling

ISO26262 Part 6 – Product development at the software level



5.1 Objectives

The objectives of this clause are:

a) to ensure a suitable and consistent software development process; and

b) to ensure a suitable software development environment.

5.4.1 When developing the software of an item, software development processes and software development environments shall be used which:

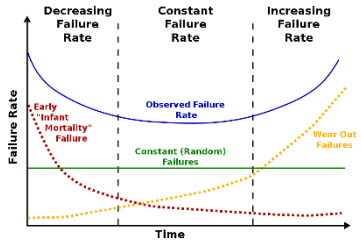
a) are suitable for developing safety-related embedded software, including methods, guidelines, languages and tools;

b) support consistency across the sub-phases of the software development lifecycle and the respective work products; and

c) are compatible with the system and hardware development phases regarding required interaction and consistency of exchange of information.

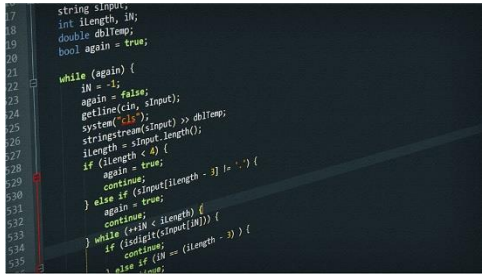
Hardware / Software Challenges

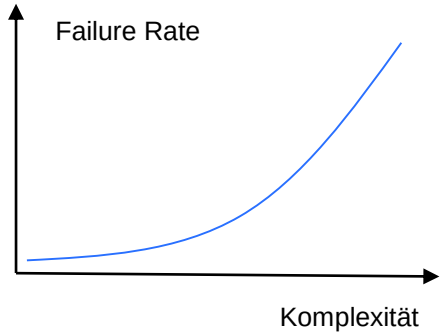
Mainly Stochastical Failures



- Reliable Components
- Redundancy
- Diagnostic Coverage

Mainly systematic Failures





- Avoid systematic errors
- Detection of stochastic (hardware) faults

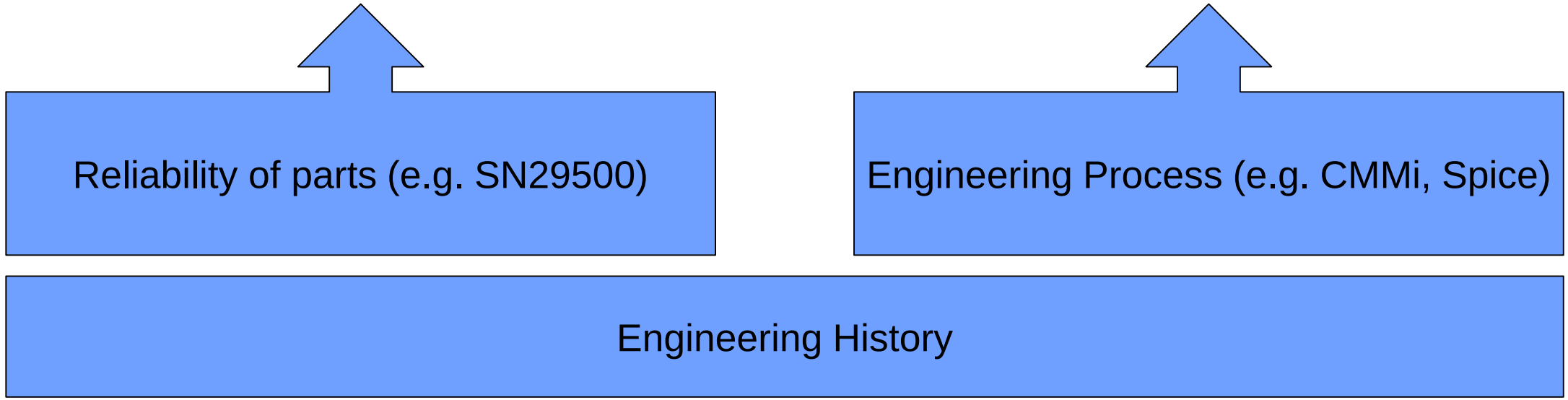
Comparison Part 5 - Hardware and Part 6 - Software

Part 5 – Hardware

- Component Reliability and Failure Rate
- Diagnostic Coverage
- Fault detection (Single Point Faults, Residual Faults, Dual Point Faults,…)
- Fault Metrics

Part 6 – Software

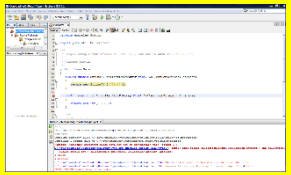
- Engineering Methodology, Processes and Tools
- Architectural Modelling
- Coding Guidelines
- Test Concepts



Before we dive into ISO26262 Part 6
Software Errors and their Root Cause – just an incomplete collection

Developer

Compiler / OS / Tools



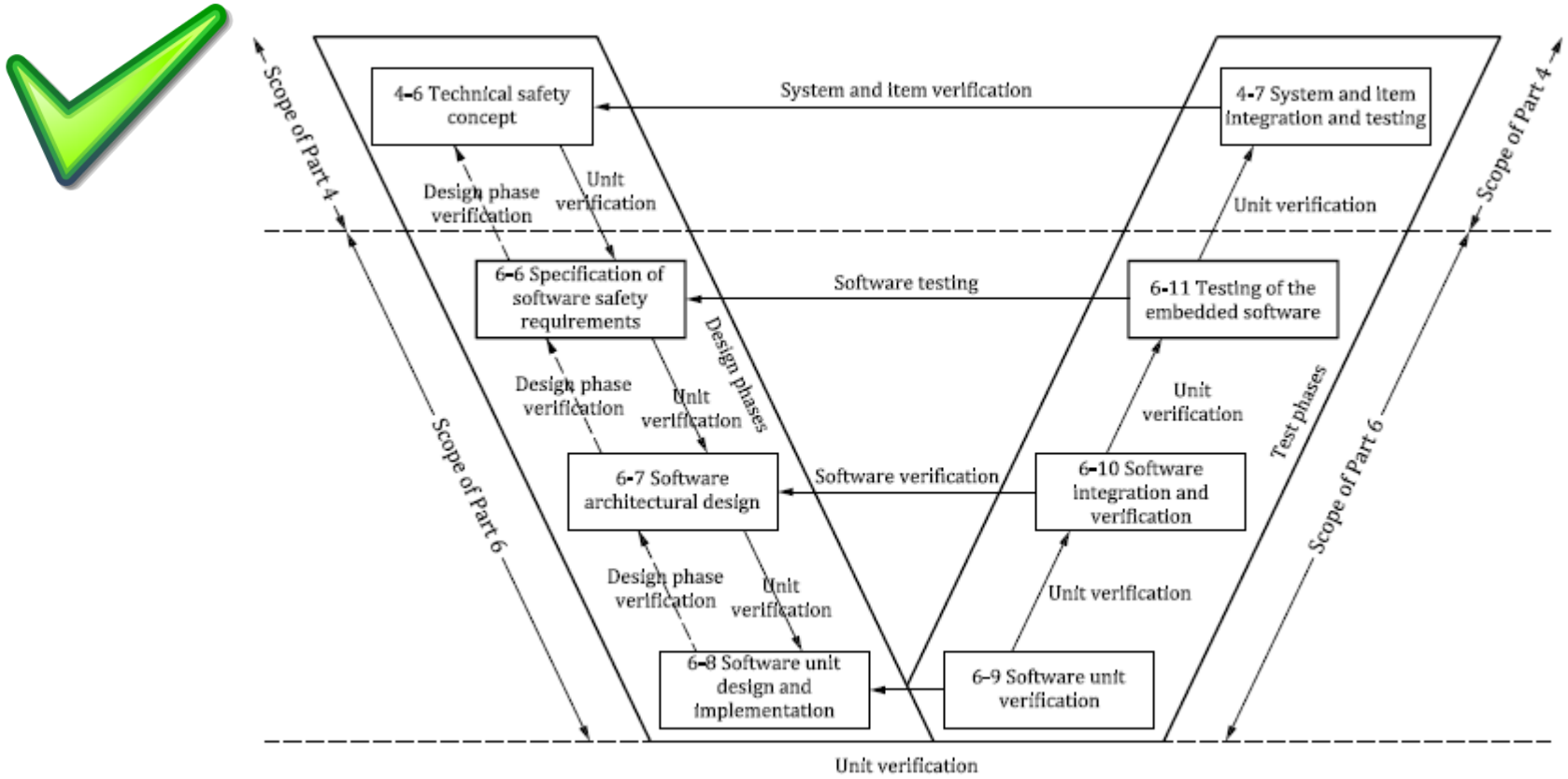
Microcontroller

- Spec not understood
- Padding / Endianess
- Forgotten Branch
- Casting
- Code rot
- Forgotten Initialisation
- Data consistency
- Blocking calls
- OS Configuration
- Missing error handling
- Driver API
- Pointer
- Semaphore usage
- …

- Optimiziation
- Data linkage
- Atomic code
- Usage of HW specific features
- MPU setting
- (Nested) Interrupt handling
- …

- Oscillating Interrupts
- Frozen Pins
- Memory Bit Toggle
- Bus errors (Timing)

Basis V-Modell



Example of Guidelines

Table 1 — Topics to be covered by modelling and coding guidelines

Topics		ASIL			
		A	B	C	D
1a	Enforcement of low complexity[a]	++	++	++	++
1b	Use of language subsets[b]	++	++	++	++
1c	Enforcement of strong typing[c]	++	++	++	++
1d	Use of defensive implementation techniques[d]	+	+	++	++
1e	Use of well-trusted design principles[e]	+	+	++	++
1f	Use of unambiguous graphical representation	+	++	++	++
1g	Use of style guides	+	++	++	++
1h	Use of naming conventions	++	++	++	++
1i	Concurrency aspects[f]	+	+	+	+

a An appropriate compromise of this topic with other requirements of this document may be required.

b The objectives of topic 1b include:

— Exclusion of ambiguously-defined language constructs which may be interpreted differently by different modellers, programmers, code generators or compilers.

— Exclusion of language constructs which from experience easily lead to mistakes, for example assignments in conditions or identical naming of local and global variables.

— Exclusion of language constructs which could result in unhandled run-time errors.

c The objective of topic 1c is to impose principles of strong typing where these are not inherent in the language.

d Examples of defensive implementation techniques:

— Verify the divisor before a division operation (different from zero or in a specific range).

— Check an identifier passed by parameter to verify that the calling function is the intended caller.

— Use the "default" in switch cases to detect an error.

e Verification of the validity of the underlying assumptions, boundaries and conditions of application may be required.

f Concurrency of processes or tasks is not limited to executing software in a multi-core or multi-processor runtime environment.

It all starts with the requirements again

- Check the safety goals and their impact on the software safety requirements
- Check possible limitations on the hardware side and add software safety features

6 Specification of software safety requirements

6.1 Objectives

The objectives of this sub-phase are:

a) to specify or refine the software safety requirements which are derived from the technical safety concept and the system architectural design specification;

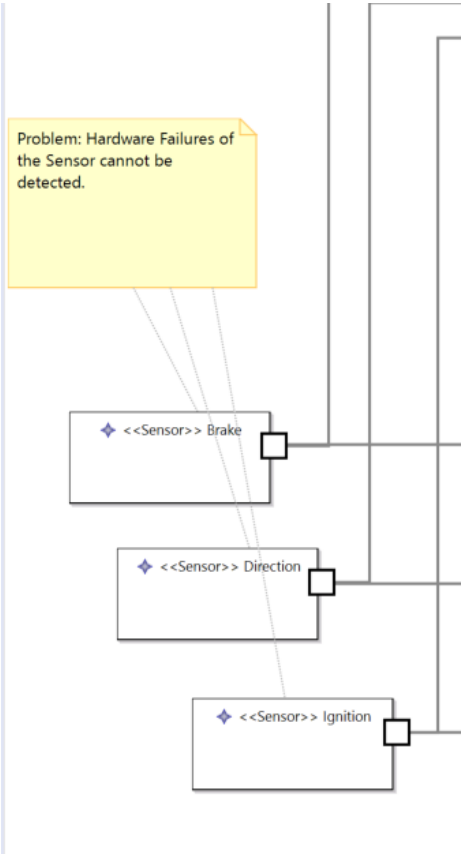
b) to define the safety-related functionalities and properties of the software required for the implementation;

c) to refine the requirements of the hardware-software interface initiated in ISO 26262-4:2018, Clause 6; and

d) to verify that the software safety requirements and the hardware-software interface requirements are suitable for software development and are consistent with the technical safety concept and the system architectural design specification.

Limitation Hardware: Button Check

- We could add a selfttest function, which is checking the button on and off state during power on
- The operation should be as simple and intuitive as possible



Derived hardware and software requirements

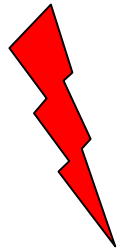
Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	• SW: Check brake pedal operation power on • SW: Ensure reliable reading of "pressed state“ • HW: Provide 1oo2 brake signal • HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	
SR2	The gas pedal signal must be reliable	C	• HW: Add resistors to detect broken / short circuit connections of gas pedal • HW: Provide 1oo2 pedal signal • HW: Use different technologies to read both channels • HW: Use inverted signals • SW: Drift of potentiometer must be detected • SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	
SR3	The car may only start after an explicit ignition sequence	QM	• SW: A clear press of the ignition button must be detected • SW: The engine may only start of the gas pedal is depressed	
SR4	The control speed value must be correctly calculated	C	• SW: Brake signal must overide gas pedal • SW: Wrong calculations of control speed value must be detetcted • SW: Supervise / Limit PID Control Parameters • SW: Driving direction may only be changed if engine is stopped • SW: Reliable monitor communication link • SW: Controlspeed will be calculated with different algo's on both controllers • SW: State Machine protection	

Derived hardware and software requirements

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	• HW: Engine power must be deactivated in case of critical failure • HW: Engine power must be controlled by independent hardware • HW: Engine current must be supervised and limited • SW: Engine Blocking must be detected • SW: Current speed must be compared with target speed • SW: Limit reverse speed • SW: Engine must be explicitly activated	
SR6	The system status must be shown to the driver	A	• SW: Provide status / error information to driver • SW: Detect freeze of OLED display	
SR7	The system integrity must be supervised	C	• HW: Provide an external watchdog, which turns of the actuator in case of a critical error • HW: Power supply must be supervised, bursts must be avoided • HW: The integrity of the controller (MCU) must be supervised • SW: The integrity of the OS must be supervised • SW: ISR bursts must be detected • SW: Low Power must be detected	

Additional Software Safety Requirements

- We need to ensure the integrity of the software on system level
  - Deadline and Control Flow Monitoring
- We need to ensure the integrity of the data and peripheral usage
  - MPU protection of data
  - MPU protection of peripherals
- We have to bring the system into a safe state in case of serious errors
  - Escalation strategy
- And we should not forget the ability to run test
  - Test triggers / mode must be part of the design



Software Architectural Design

7 Software architectural design

7.1 Objectives

The objectives of this sub-phase are:

a) to develop a software architectural design that satisfies the software safety requirements and the other software requirements;

b) to verify that the software architectural design is suitable to satisfy the software safety requirements with the required ASIL; and

c) to support the implementation and verification of the software.

7.2 General

The software architectural design represents the software architectural elements and their interactions in a hierarchical structure. Static aspects, such as interfaces between the software components, as well as dynamic aspects, such as process sequences and timing behaviour, are described.

NOTE The software architectural design is not necessarily limited to one microcontroller or ECU. The software architecture for each microcontroller is also addressed by this sub-clause.

A software architectural design is able to satisfy both the software safety requirements as well as the other software requirements. Hence, in this sub-phase, safety-related and non-safety-related software requirements are handled within one development process.

The software architectural design provides the means to implement both the software requirements and the software safety requirements with the required ASIL and to manage the complexity of the detailed design and the implementation of the software.

Architectural Description / Modelling

Focus

- Natural Language (Prose) +
- Informal/Semiformal Notations

Examples for semiformal notations

- UML, SysML
- Simulink, Stateflow
- Autosar

7.4 Requirements and recommendations

7.4.1 To avoid systematic faults in the software architectural design and in the subsequent development activities, the description of the software architectural design shall address the following characteristics supported by notations for software architectural design as listed in Table 2:

a) comprehensibility;

b) consistency;

c) simplicity;

d) verifiability;

e) modularity;

f) abstraction;

NOTE Abstraction can be supported by using hierarchical structures, grouping schemes or views to cover static, dynamic or deployment aspects of an architectural design.

g) encapsulation; and

h) maintainability.

Table 2 — Notations for software architectural design

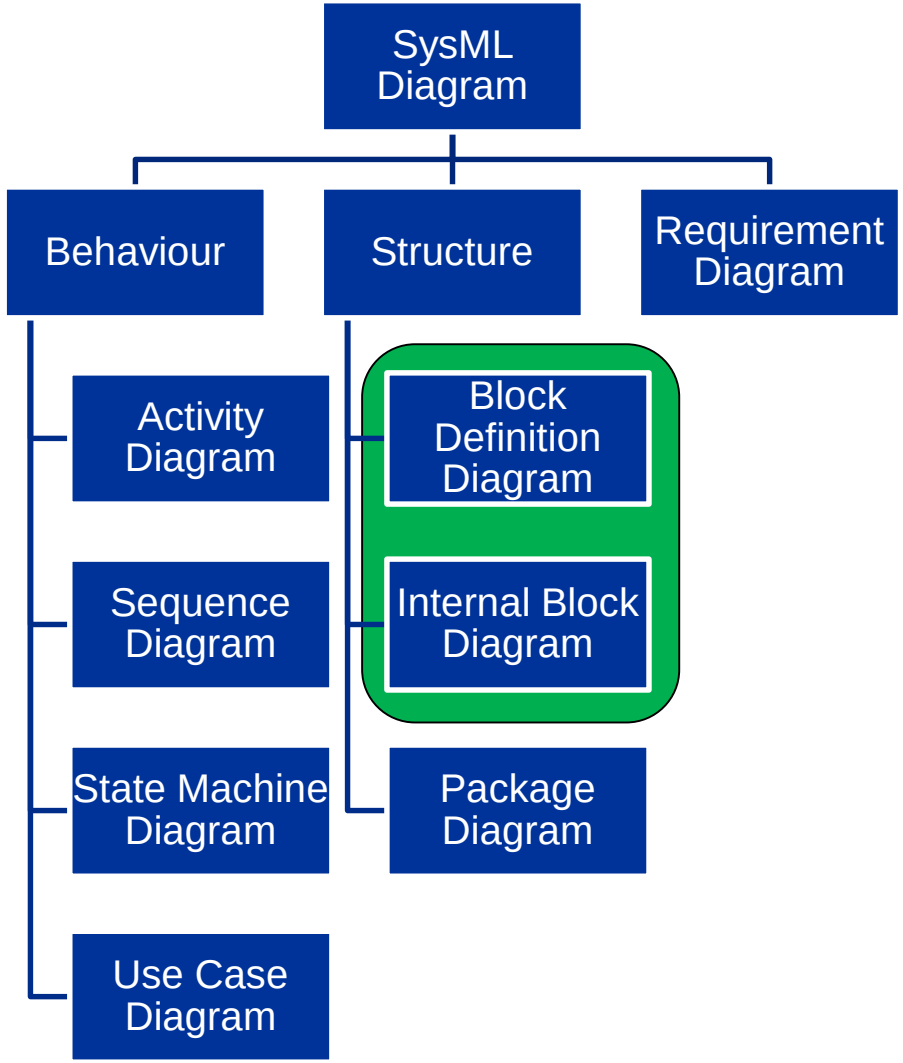
	Notations	ASIL A	ASIL B	ASIL C	ASIL D
1a	Natural language[a]	++	++	++	++
1b	Informal notations	++	++	+	+
1c	Semi-formal notations[b]	+	+	++	++
1d	Formal notations	+	+	+	+

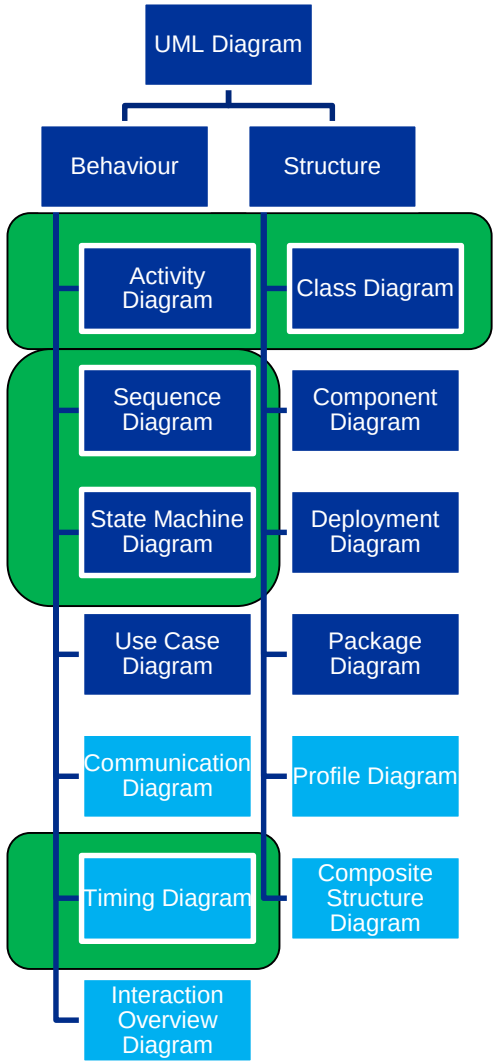
[a] Natural language can complement the use of notations for example where some topics are more readily expressed in natural language or providing explanation and rationale for decisions captured in the notation.

[b] Semi-formal notations can include pseudocode or modelling with UML®, SysML®, Simulink® or Stateflow®.

NOTE UML®, SysML®, Simulink® and Stateflow® are examples of suitable products available commercially. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO of these products.

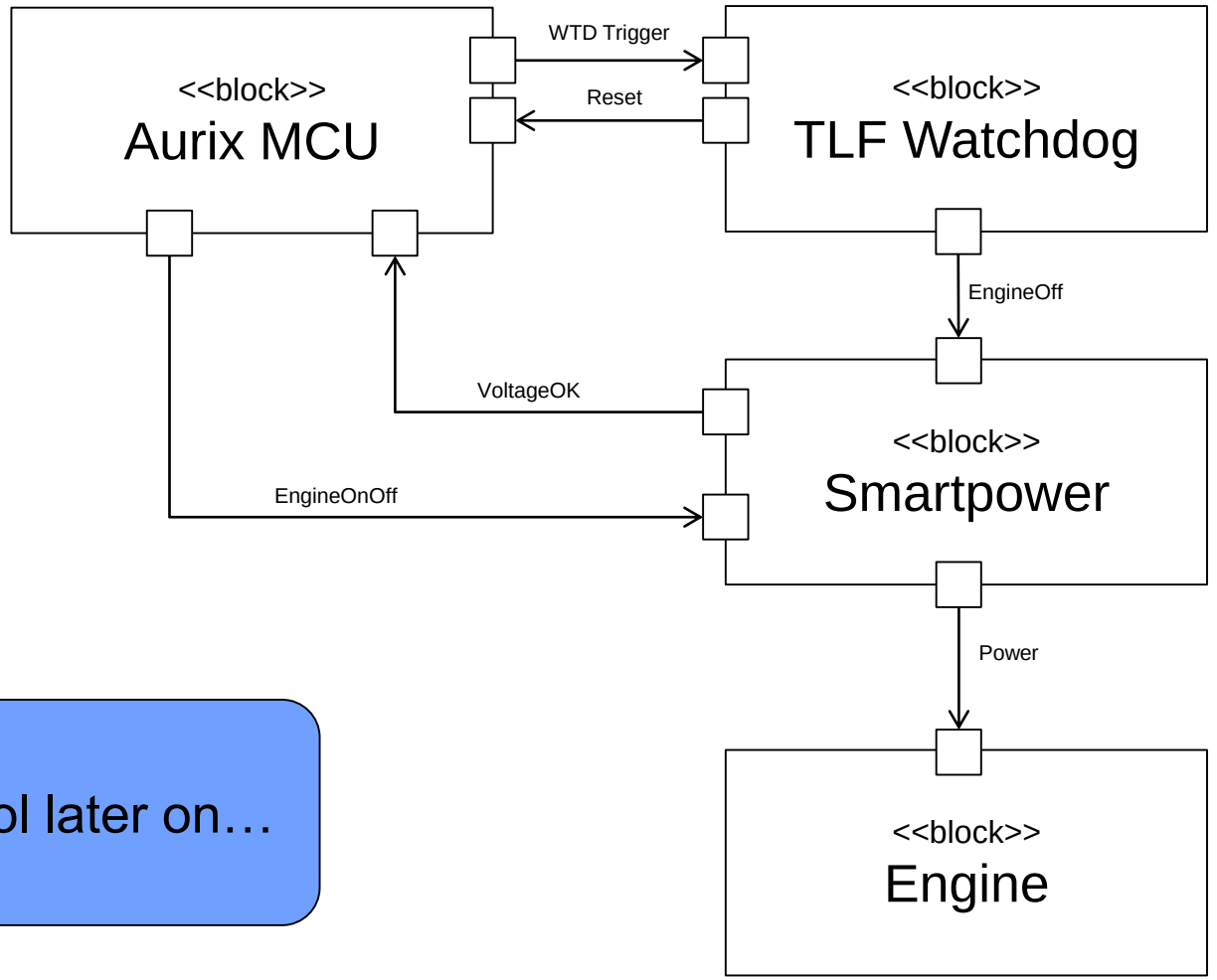
UML and SysML





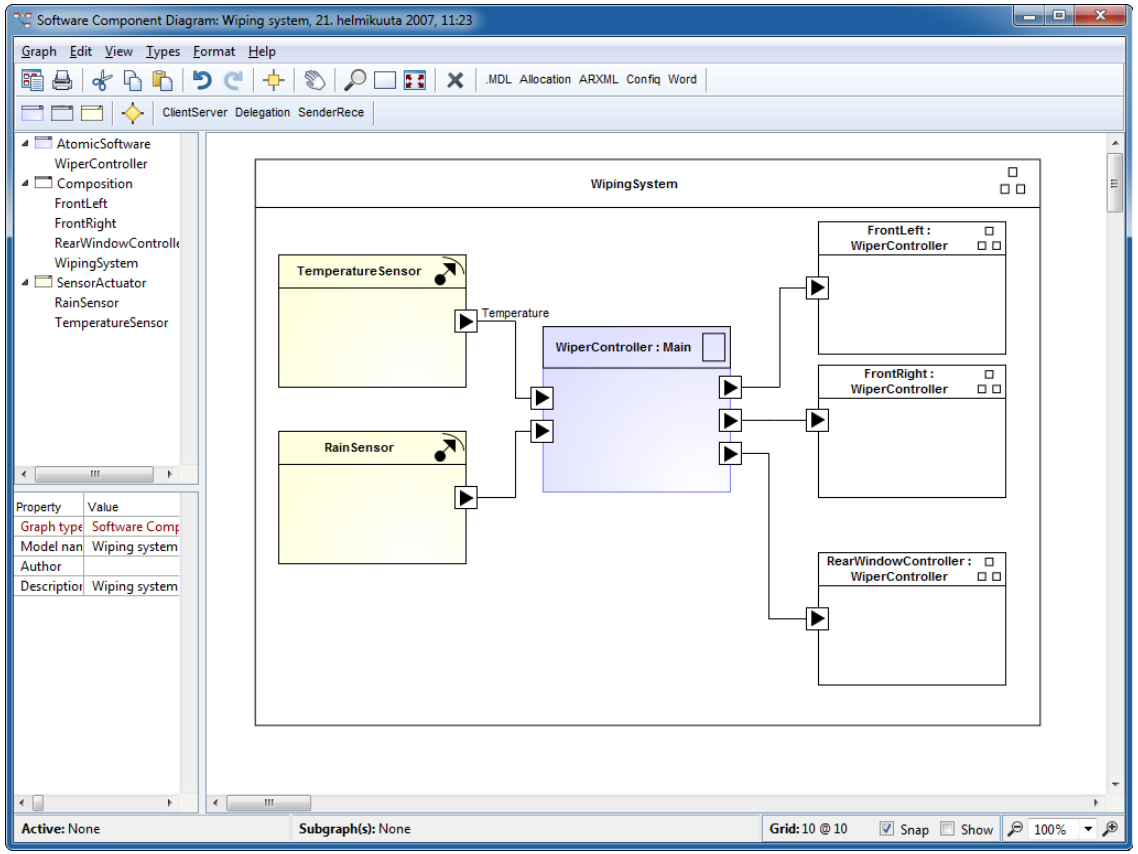
Example: SysML Block Diagram

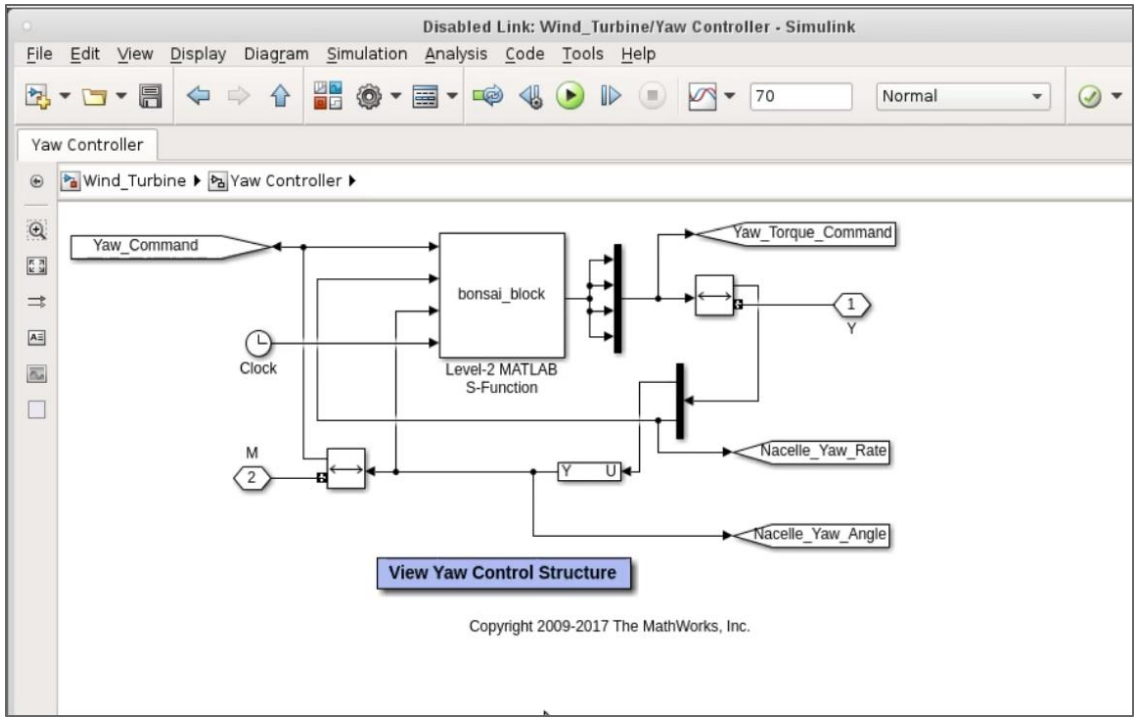
- Very flexible applicability
- Easy to understand
- Represents structure and data flow



We will use this idea for our modelling tool later on…

Autosar / Matlab Simulink





Example Formal Methods

- Provision of very strict and precise semantics

MACHINE
StudentCouncil(*limit*)

CONSTRAINTS
$limit \in \mathcal{N}_1$

SETS
STUDENT

VARIABLES
council, *president*

INVARIANT
$council \in \mathcal{P}(STUDENT) \wedge \mathbf{card}(council) \leq \text{limit} \wedge$
$president \in \text{STUDENT} \wedge$
$(council \neq \phi \Rightarrow president \in council)$

INITIALIZATION
$council := \phi \;||$
$president :\in STUDENT$

OPERATIONS
AddStudent(*nn*) =
PRE
$nn \in STUDENT \wedge nn \notin council \wedge \mathbf{card}(council) < limit$
THEN
IF $council = \phi$ **THEN**
$president := nn$
END ||
$council := council \cup \{nn\}$
END;
[...]

Figure 2.1. Example of the *B* notatation taken from the book *Specification of Software Systems* [2]. The example shows a specification of a student council. Elements from the set of students can be members of the council. If the council has any members, the president must be one of them. Double bars in B does not mean "logical or"; instead it means that the statements on both sides are executed in parallel, i.e. at the same time. The symbol $\mathcal{N}_1$ denotes integers greater than zero.

Design Principles

Table 3 — Principles for software architectural design

	Principles	ASIL A	ASIL B	ASIL C	ASIL D
1a	Appropriate hierarchical structure of the software components	++	++	++	++
1b	Restricted size and complexity of software components[a]	++	++	++	++
1c	Restricted size of interfaces[a]	+	+	+	++
1d	Strong cohesion within each software component[b]	+	++	++	++
1e	Loose coupling between software components[b,c]	+	++	++	++
1f	Appropriate scheduling properties	++	++	++	++
1g	Restricted use of interrupts[a,d]	+	+	+	++
1h	Appropriate spatial isolation of the software components	+	+	+	++
1i	Appropriate management of shared resources[e]	++	++	++	++

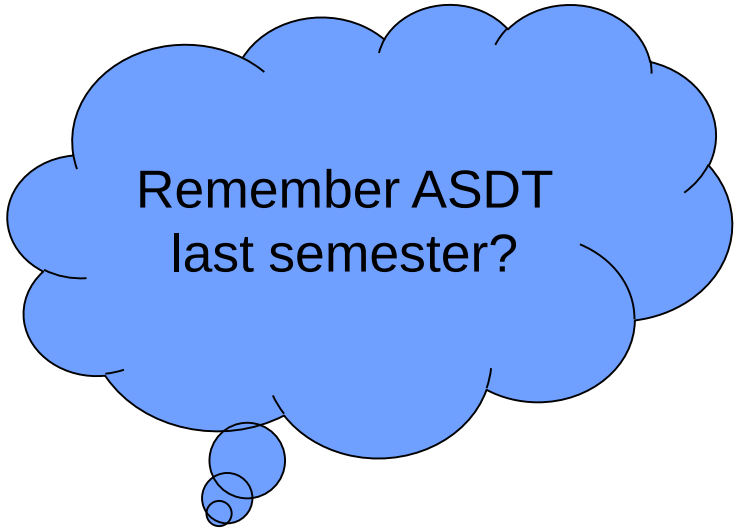
[a] In principles 1b, 1c, and 1g "restricted" means to minimize in balance with other design considerations.

[b] Principles 1d and 1e can, for example, be achieved by separation of concerns which refers to the ability to identify, encapsulate, and manipulate those parts of software that are relevant to a particular concept, goal, task, or purpose.

[c] Principle 1e addresses the management of dependencies between software components.

[d] Principle 1g can include minimizing the number, or using interrupts with a clear priority, in order to achieve determinism.

[e] Principle 1i applies for shared hardware resources as well as shared software resources in the case of coexistence. Such resource management can be implemented in software or hardware and includes safety mechanisms and/or process measures that prevent conflicting access to shared resources as well as mechanisms that detect and handle conflicting access to shared resources.



Good design principles
Control flow
Freedom from Interference

Error Detection

- Range checks of input and output data;
- Plausibility check (e.g. using a reference model of the desired behaviour, assertion checks, or comparing signals from different sources);
- Detection of data errors (e.g. error detecting codes and multiple data storage);

- Monitoring of program execution by an external element such as an ASIC or another software element performing a watchdog function. Monitoring can be logical or temporal monitoring or both;
- Temporal monitoring of program execution;

- Diverse redundancy in the design; or
- Access violation control mechanisms implemented in software or hardware concerned with granting or denying access to safety-related shared resources.

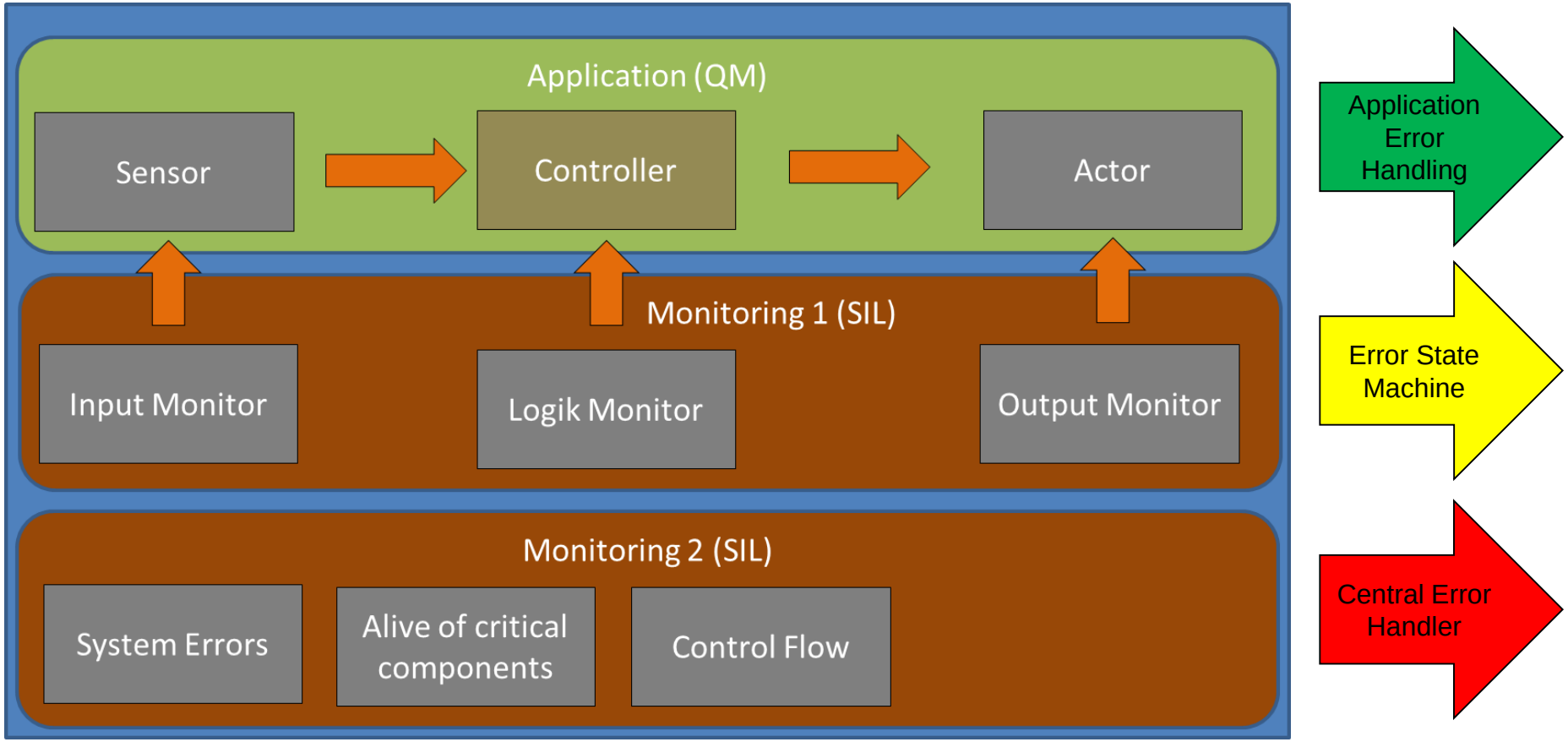
Error Handling

- Deactivation in order to achieve and maintain a safe state;
- Static recovery mechanism (e.g. recovery blocks, backward recovery, forward recovery and recovery through repetition);
- Graceful degradation by prioritizing functions to minimize the adverse effects of potential failures on functional safety;

- Homogenous redundancy in the design, which focuses primarily on controlling the effects of transient faults or random faults in the hardware on which a similar software is executed (e.g. temporal redundant execution of software);
- Diverse redundancy in the design which implies dissimilar software in each parallel path and focuses primarily on the prevention or control of systematic faults in the software;

- Correcting codes for data; or
- Access permission management implemented in software or hardware concerned with granting or denying access to safety-related shared resources.

3 Layer Safety Architecture



Ressource estimation

- Worst case execution time

- Memory
  - Global
  - Stack (per task)
  - Heap (per task)
  - Nonvolatile memory (Flash, EEPROM)

- Communication ressources
  - Frequency of calling
  - Amount of data
  - Bandwith

Stress or overload situations are always critical!

Design verification

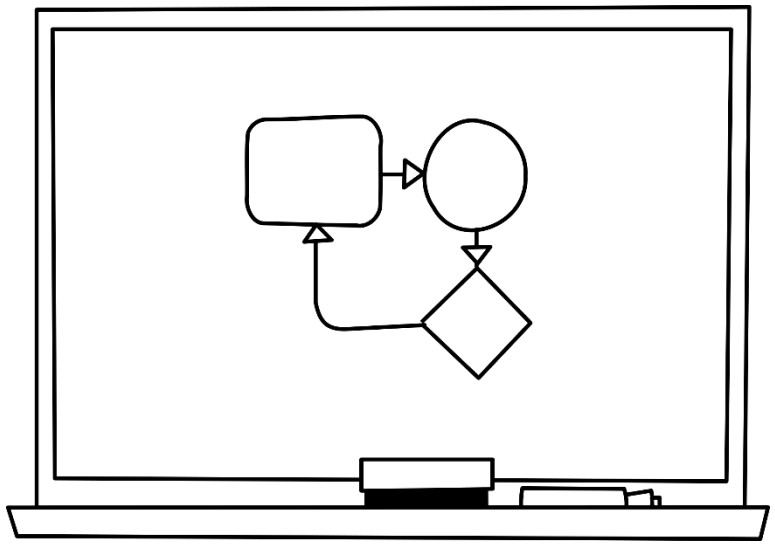
Table 4 — Methods for the verification of the software architectural design

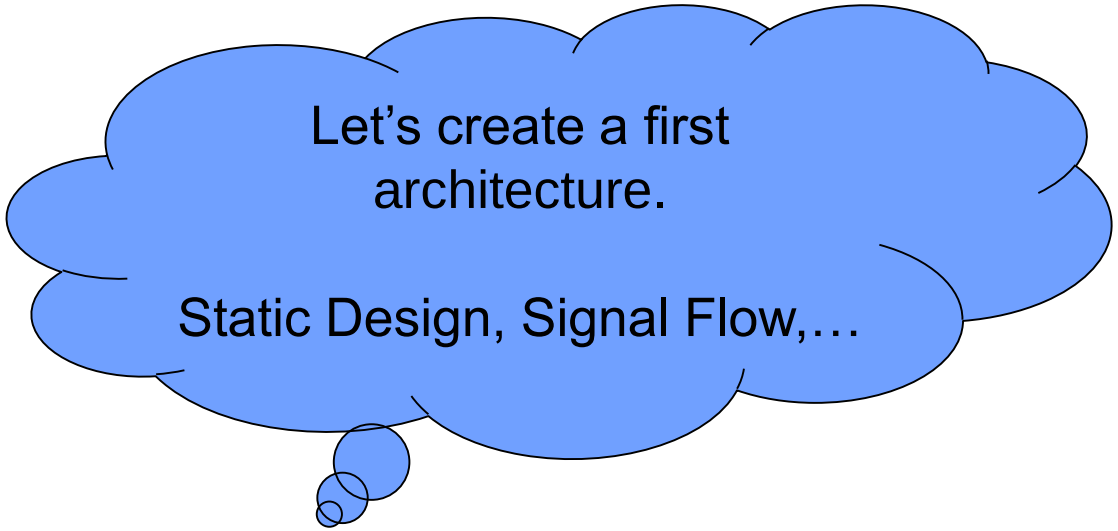
Methods		ASIL			
		A	B	C	D
1a	Walk-through of the design[a]	++	+	o	o
1b	Inspection of the design[a]	+	++	++	++
1c	Simulation of dynamic behaviour of the design	+	+	+	++
1d	Prototype generation	o	o	+	++
1e	Formal verification	o	o	+	+
1f	Control flow analysis[b]	+	+	++	++
1g	Data flow analysis[b]	+	+	++	++
1h	Scheduling analysis	+	+	++	++

[a] In the case of model-based development, these methods can also be applied to the model.

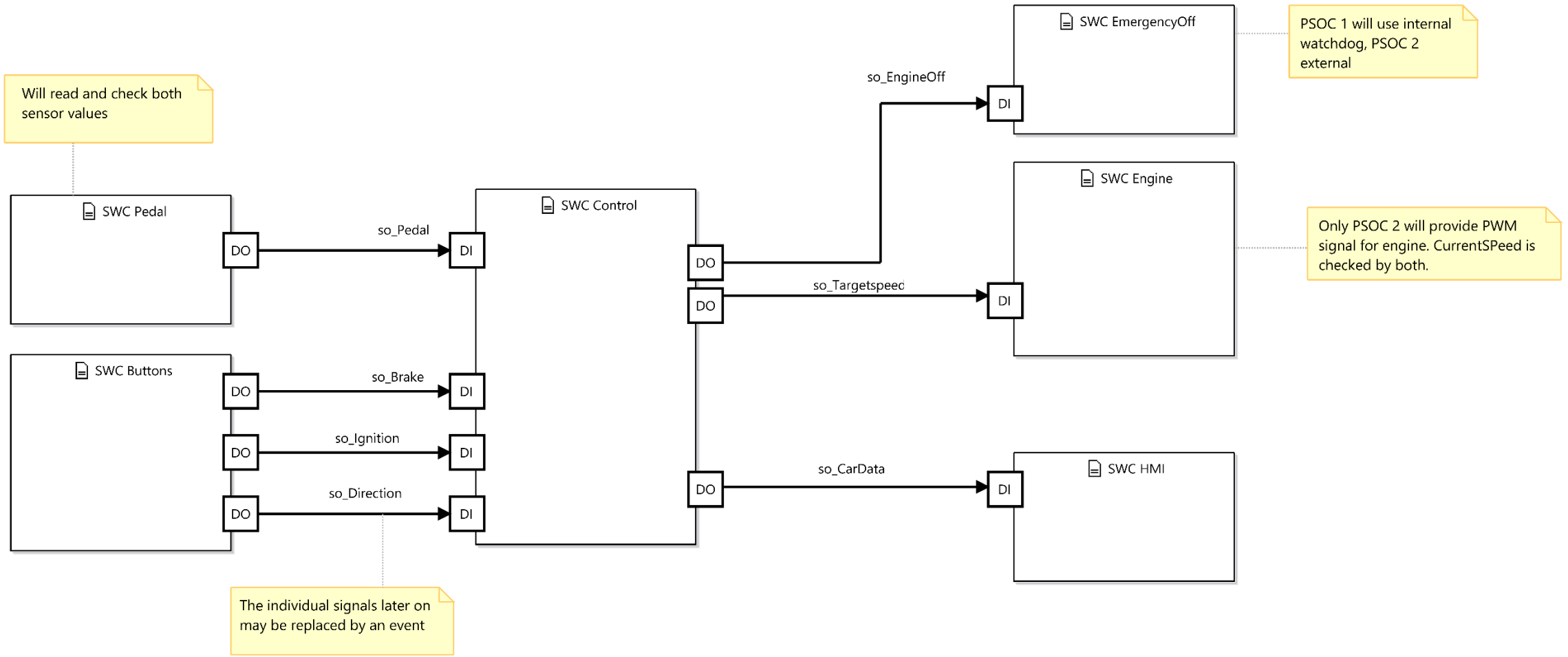
[b] Control and data flow analysis can be limited to safety-related components and their interfaces.

Software Architecture





A first rough software architecture – single channel



Unit (Class) Design and Implementation

Just do it!

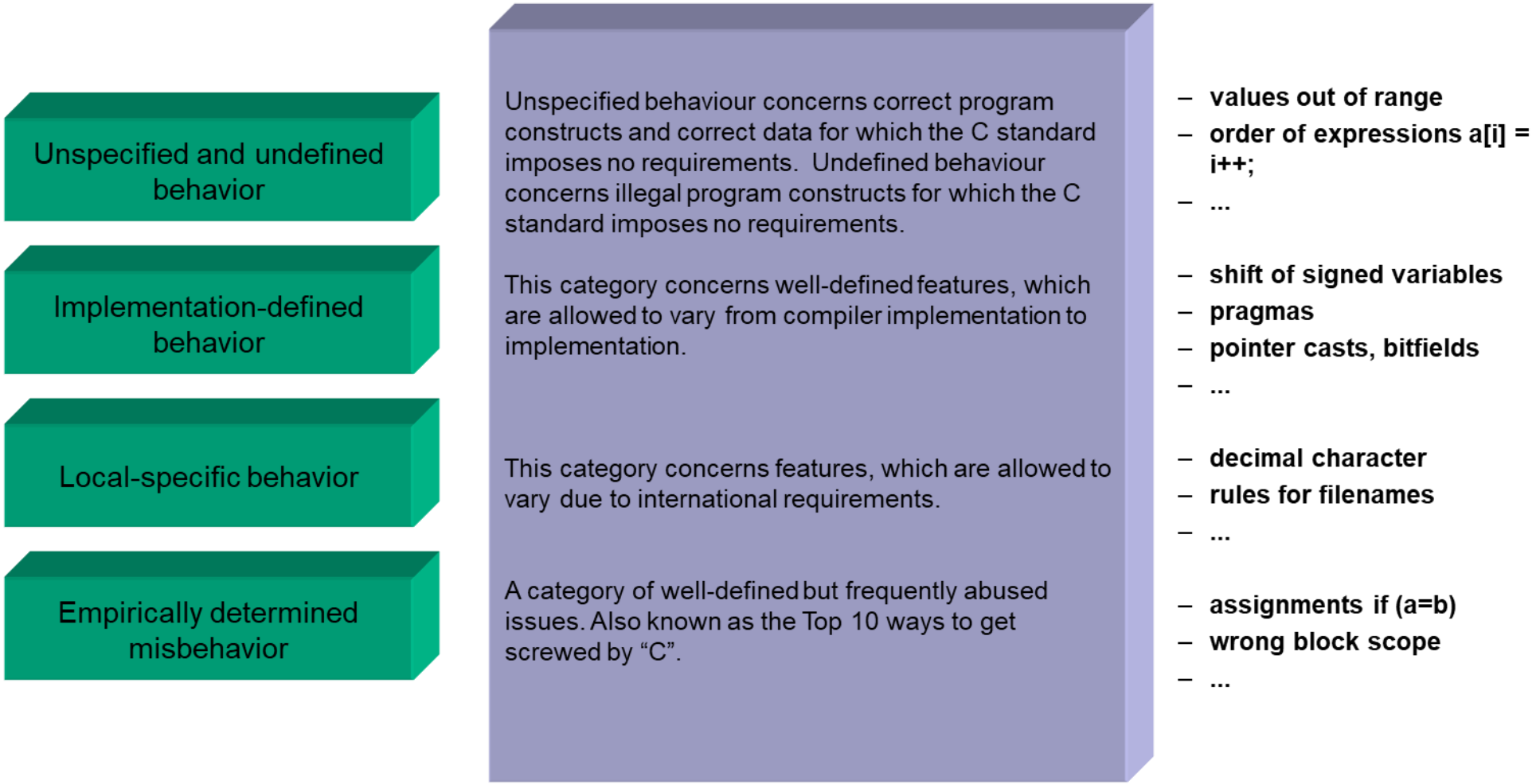
Table 6 — Design principles for software unit design and implementation

Principle		ASIL			
		A	B	C	D
1a	One entry and one exit point in subprograms and functions[a]	++	++	++	++
1b	No dynamic objects or variables, or else online test during their creation[a]	+	++	++	++
1c	Initialization of variables	++	++	++	++
1d	No multiple use of variable names[a]	++	++	++	++
1e	Avoid global variables or else justify their usage[a]	+	+	++	++
1f	Restricted use of pointers[a]	+	++	++	++
1g	No implicit type conversions[a]	+	++	++	++
1h	No hidden data flow or control flow	+	++	++	++
1i	No unconditional jumps[a]	++	++	++	++
1j	No recursions	+	+	++	++

[a] Principles 1a, 1b, 1d, 1e, 1f, 1g and 1i may not be applicable for graphical modelling notations used in model-based development.

NOTE For the C language, MISRA C (see Reference [3]) covers many of the principles listed in Table 6.

To make things a bit more interesting…



C/C++ Standard Types

Compatible type table from Standard (types in each row are compatible)
void
char
signed char
unsigned char
short, signed short, short int, signed short int
unsigned short, unsigned short int
int, signed, signed int, or no type specifiers
unsigned, unsigned int
long, signed long, long int, signed long int
unsigned long, unsigned long int
float
double
long double

What the standard says

- No two signed integer types shall have the same rank, even if they have the same representation.
- The rank of a signed integer type shall be greater than the rank of any signed integer type with less precision.
- The rank of long long int shall be greater than the rank of long int, which shall be greater than the rank of int, which shall be greater than the rank of short int, which shall be greater than the rank of signed char.

Use POSIX Types:

uint8_t, sint8_t, uint16_t, sint16_t,…

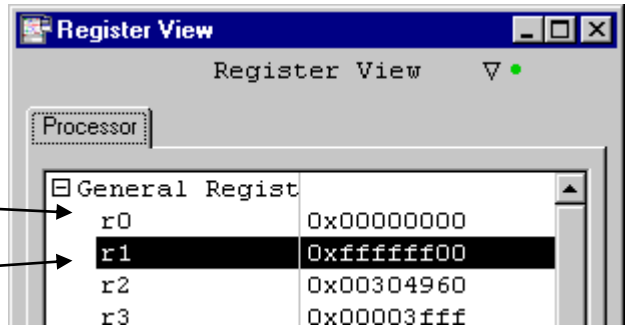
Assignment, Promotion, Balancing
How the CPU performs operations

```
unsigned char u8_v1;
unsigned char u8_v2;
u8_v1 = ~0xff;
u8_v2 = 0xff;

if (u8_v1 == ~0xff)
{
   // Is this part executed?
}

//what about
if (u8_v1 == ~u8_v2)...?
```

assignment, promotion

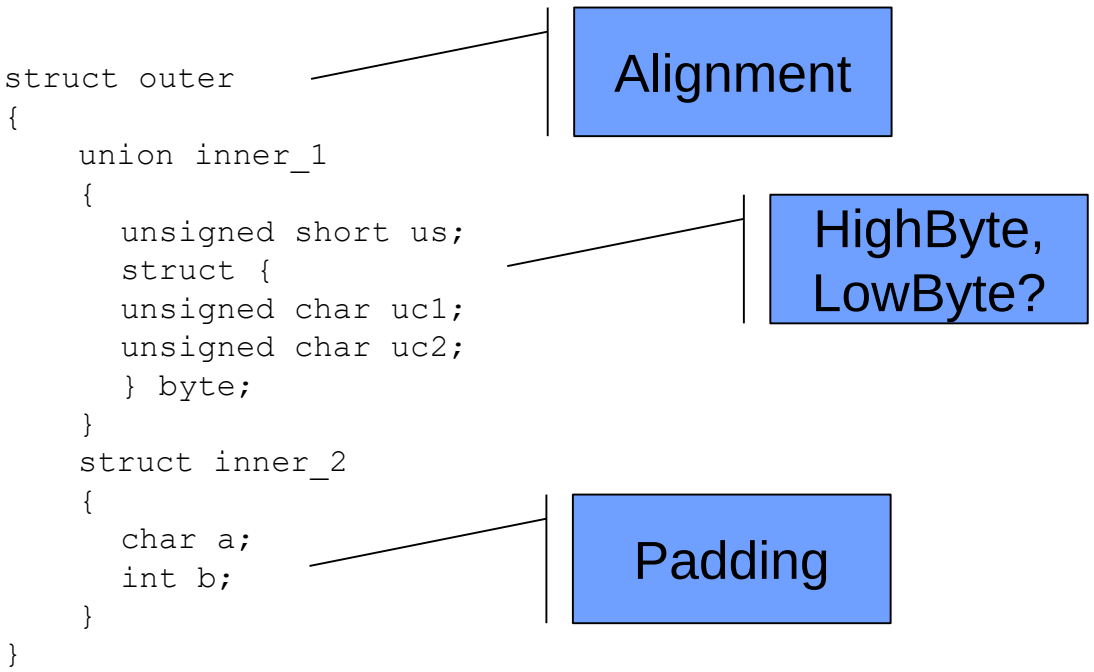



```
        #include <stdio.h>

        void main()
1       {
2       unsigned char   u_ch_v1;
    •   0x100098  main:         e92d4300   STMDB     SP!, {R8,R9,LR}
3       u_ch_v1 = ~0xff;
    •   0x10009c  main+0x4:     e3a00000   MOV       R0, 0
4
5
6   ➡   if (u_ch_v1 == ~0xff)
    STOP 0x1000a0  main+0x8:     e3e010ff   MVN       R1, 0xff
    STOPPED 0x1000a4  main+0xc:  e1500001   CMP       R0, R1
    •   0x1000a8  main+0x10:    1a000001   BNE       0x4(main+0x1c (0x1000b4))
7       {
8           printf("Just did the trick\n");     /* NEVER EXECUTED */
    •   0x1000ac  main+0x14:    e59f0008   LDR       R0,[PC,8](0x1000bc)
    •   0x1000b0  main+0x18:    eb00000a   BL        0x28(printf (0x1000e0))
9       }
10
```

Structs, Alignsments, Endianess, Badding, Bitorder

- The compiler has quite some freedom to allocate data in memory
- Typically it will try to optimize the access to this data



Pointer

Who enjoys pointers?

- Complex syntax
- Hard to understand (human brain seems to have some incompatibilities)
- Memory needs to be allocated and deallocated by the user
- Different lifetime of allocated memory and pointer variable
- Garbage collection and segmented memory

Control Flow

- Nested IF structures are hard to read - use indentations and { } for all statements and always implement the easiest logic
- Conditions are forgotten - always provide catch all / default clauses
- Forgotten BREAK statements in SWITCH statements
- Expressions in conditions
- Floats in conditions
- Changing control variables inside the body of a FOR statement
- Using GOTO statements
- Using implicite comparison, e.g. if (func()) ...
- Unclear boolean expressions (On, Off) or redefinition of expressions
- Inverted HW-registers,e.g. 0 = ON, 1 = OFF
- …

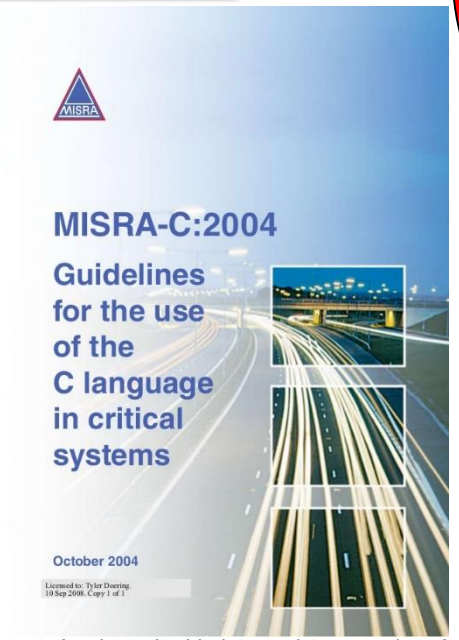
Defensive Programming and Misra

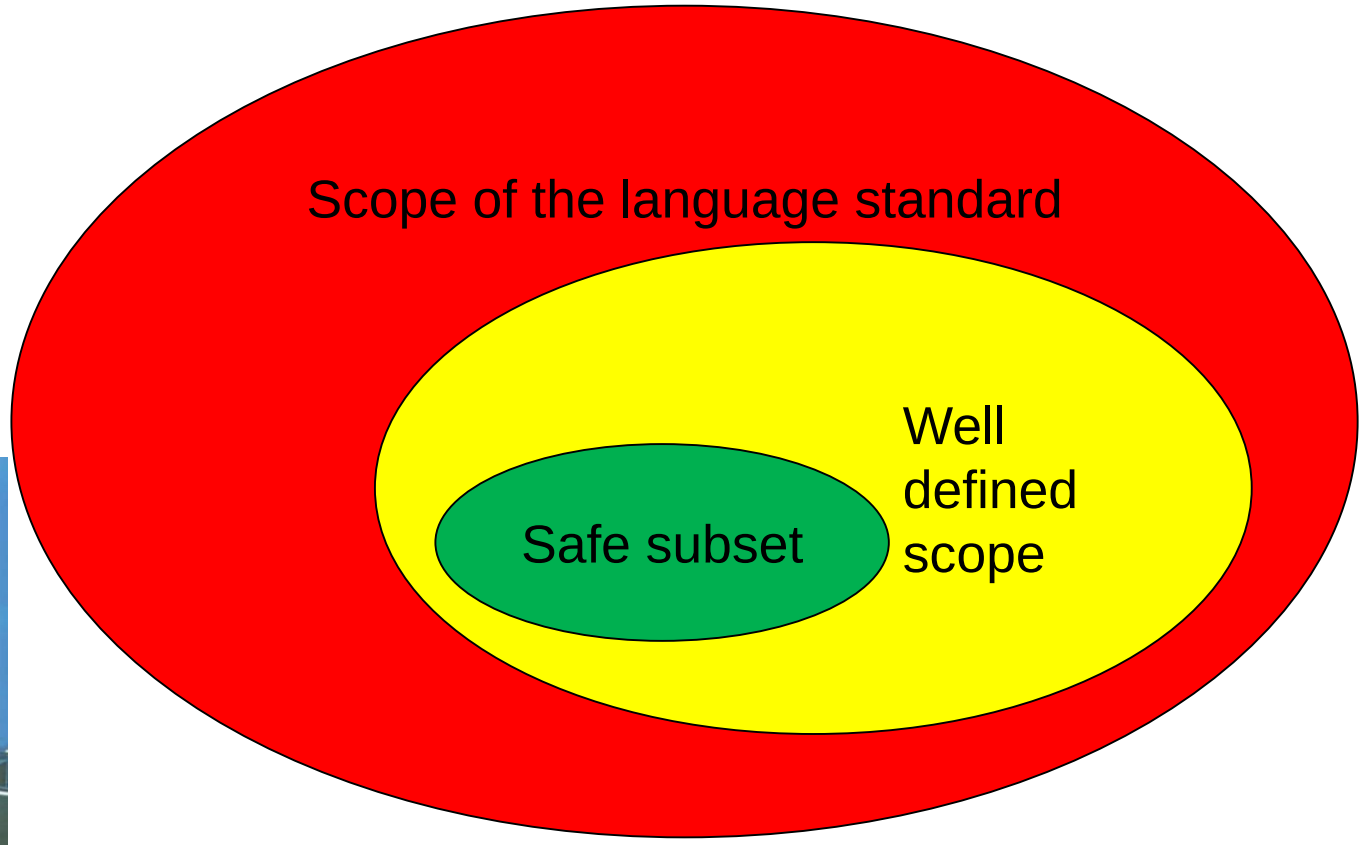
MISRA C

It is a software development standard for the C programming language. Its aims are to facilitate code portability and reliability in the context of embedded systems, specifically those systems programmed in ANSI C.

Standards and reliability

Even though there is not a *MISRA certification process*, MISRA guidelines are thoroughly followed, expecially in automotive industry, as they represent one of the most popular standards for developing secure software.





Example MISRA

Rule 6.3 (advisory): ***typedefs* that indicate size and signedness should be used in place of the basic numerical types.**

The basic numerical types of *signed* and *unsigned* variants of *char*, *int*, *short*, *long* and *float*, *double* should not be used, but specific-length *typedefs* should be used. Rule 6.3 helps to clarify the size of the storage, but does not guarantee portability because of the asymmetric behaviour of integral promotion. See discussion of integral promotion — section 6.10. It is still important to understand the integer size of the implementation.

Programmers should be aware of the actual implementation of the *typedefs* under these definitions.

For example, the ISO (POSIX) *typedefs* as shown below are recommended and are used for all basic numerical and character types in this document. For a 32-bit integer machine, these are as follows:

```
typedef          char   char_t;
typedef signed   char   int8_t;
typedef signed   short  int16_t;
typedef signed   int    int32_t;
typedef signed   long   int64_t;
typedef unsigned char   uint8_t;
typedef unsigned short  uint16_t;
typedef unsigned int    uint32_t;
typedef unsigned long   uint64_t;
typedef          float  float32_t;
typedef          double float64_t;
typedef long     double float128_t;
```

typedefs are not considered necessary in the specification of bit-field types.

- **Type widening in integral promotion**: The type in which integral expressions are evaluated depends on the type of the operands after any integral promotion. It is always possible to multiply two 8-bit values and access a 16-bit result if the magnitude requires it. It is sometimes, but not always, possible to multiply two 16-bit values and retrieve a 32-bit result. This is a dangerous inconsistency in the C language and in order to avoid confusion it is safer never to rely on the widening type afforded by integral promotion. Consider the following example:

```
uint16_t u16a = 40000;   /* unsigned short / unsigned int ?   */
uint16_t u16b = 30000;   /* unsigned short / unsigned int ?   */
uint32_t u32x;           /* unsigned int / unsigned long  ?   */

u32x = u16a + u16b;      /* u32x = 70000 or 4464 ?            */
```

The expected result is presumably 70000, but the value assigned to `u` will in practice depend on the implemented size of an *int*. If the implemented size of an *int* is 32 bits, the addition will occur in 32-bit signed arithmetic and the correct value will be stored. If the implemented size of an *int* is only 16 bits, the addition will take place in 16-bit unsigned arithmetic, wraparound will occur and will yield the value 4464 (70000 % 65536). Wraparound in unsigned arithmetic is well defined and may even be intended; but there is potential for confusion.

Unit Verification – Systematic Review AND Test

Table 7 — Methods for software unit verification

	Methods	ASIL A	ASIL B	ASIL C	ASIL D
1a	Walk-through[a]	++	+	o	o
1b	Pair-programming[a]	+	+	+	+
1c	Inspection[a]	+	++	++	++
1d	Semi-formal verification	+	+	++	++
1e	Formal verification	o	o	+	+
1f	Control flow analysis[b, c]	+	+	++	++
1g	Data flow analysis[b, c]	+	+	++	++
1h	Static code analysis[d]	++	++	++	++
1i	Static analyses based on abstract interpretation[e]	+	+	+	+
1j	Requirements-based test[f]	++	++	++	++
1k	Interface test[g]	++	++	++	++

[a] For model-based development these methods are applied at the model level, if evidence is available that justifies confidence in the code generator used.

[b] Methods 1f and 1g can be applied at the source code level. These methods are applicable both to manual code development and to model-based development.

[c] Methods 1f and 1g can be part of methods 1e, 1h or 1i.

[d] Static analyses are a collective term which includes analysis such as searching the source code text or the model for patterns matching known faults or compliance with modelling or coding guidelines.

[e] Static analyses based on abstract interpretation are a collective term for extended static analysis which includes analysis such as extending the compiler parse tree by adding semantic information which can be checked against violation of defined rules (e.g. data-type problems, uninitialized variables), control-flow graph generation and data-flow analysis (e.g. to capture faults related to race conditions and deadlocks, pointer misuses) or even meta compilation and abstract code or model interpretation.

[f] The software requirements at the unit level are the basis for this requirements-based test. These include the software unit design specification and the software safety requirements allocated to the software unit.

[g] This method can be used to provide evidence for the integrity of used and exchanged data.

[h] In the context of software unit testing, fault injection test means to modify the tested software unit (e.g. introduce faults into the software) for the purposes described in 9.4.2. Such modifications include injection of arbitrary faults (e.g. by corrupting values of variables, by introducing code mutations, or by corrupting values of CPU registers).

[i] Some aspects of the resource usage evaluation can only be performed properly when the software unit tests are executed on the target environment or if the emulator for the target processor adequately supports resource usage tests.

[j] This method requires a model that can simulate the functionality of the software units. Here, the model and code are stimulated in the same way and results compared with each other.

EXAMPLE In the case of model-based design results of non-floating-point operations can be compared.

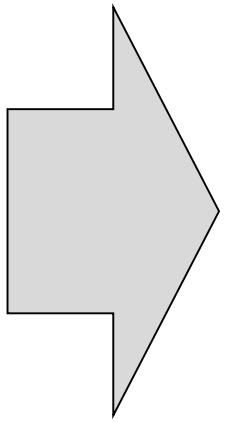
	Methods	ASIL A	ASIL B	ASIL C	ASIL D
1l	Fault injection test[h]	+	+	+	++
1m	Resource usage evaluation[i]	+	+	+	++
1n	Back-to-back comparison test between model and code, if applicable[j]	+	+	++	++

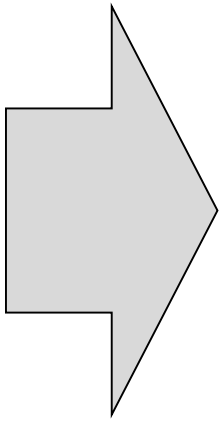
	Methods	ASIL A	ASIL B	ASIL C	ASIL D
1a	Analysis of requirements	++	++	++	++
1b	Generation and analysis of equivalence classes[a]	+	++	++	++
1c	Analysis of boundary values[b]	+	++	++	++
1d	Error guessing based on knowledge or experience[c]	+	+	+	+

	Methods	ASIL A	ASIL B	ASIL C	ASIL D
1a	Statement coverage	++	++	+	+
1b	Branch coverage	+	++	++	++
1c	MC/DC (Modified Condition/Decision Coverage)	+	+	+	++

Static Code Analysis

```
for (int i = 0; i < 10; i++)
{
    if (m_pbuffer[i] != 0)
       m_pBuffer[i]->print();
}
```

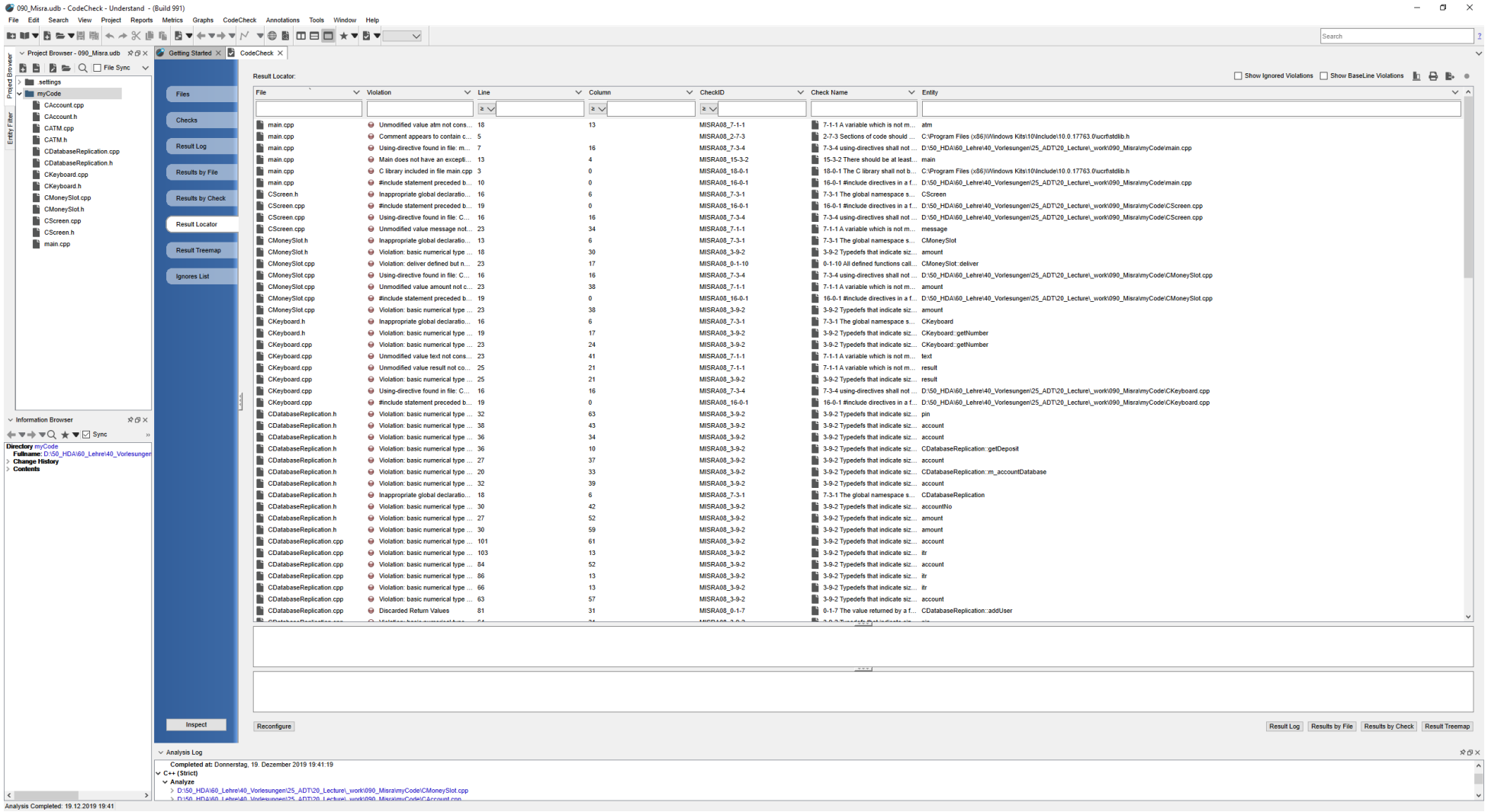




➔ **Avoid MAGIC NUMBERS**

```
for (int i = 0; i < 10; i++)
{
    if (m_pbuffer[i] != 0)
       m_pBuffer[i]->print();
}
```

Example Static Code Analysis



Wrap-Up

- Software Safety Requirements
  - Safety Goals
  - Risk mitigation for hardware faults (selftest)
  - Standard requirements (freedom from interference, alive monitoring, control flow…)

- Architecture
  - Explicit design / model required
  - Traceability towards safety requirements must be proven
  - Well structured design is a must!

- Coding
  - KISS again – simple interfaces,..
  - Coding Guidelines
  - Reviews and static code analysis
  - Test specification (traceability)
  - Black box and white box test